

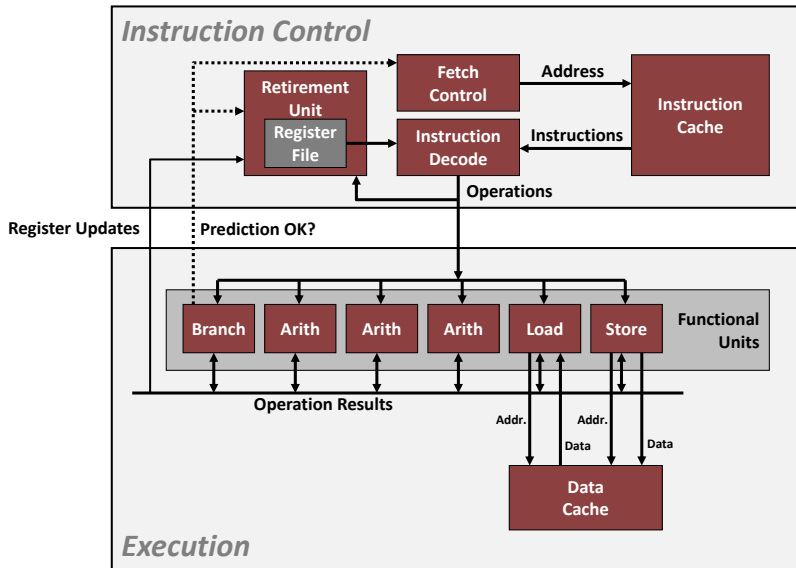
Predykcja skoków

ASK 2026

Instytut Informatyki
Uniwersytet Wrocławski

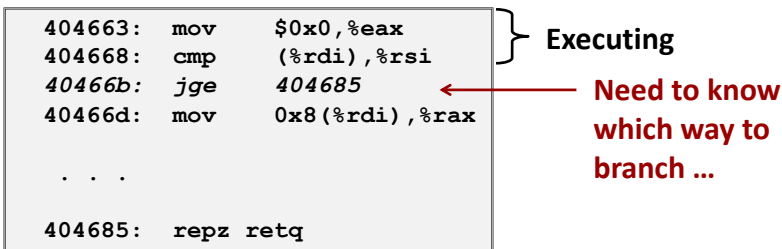
15 kwietnia 2026

Modern CPU Design



Branches Are A Challenge

- **Instruction Control Unit** must work well ahead of **Execution Unit** to generate enough operations to keep EU busy



If the CPU has to wait for the result of the `cmp` before continuing to fetch instructions, may waste tens of cycles doing nothing!

Branch Prediction

■ *Guess* which way branch will go

- Begin executing instructions at predicted position
- But don't actually modify register or memory data

```
404663:  mov    $0x0,%eax
404668:  cmp    (%rdi),%rsi
40466b:  jge    404685
40466d:  mov    0x8(%rdi),%rax

. . .

404685:  repz   retq
```

Predict Taken

**Continue
Fetching
Here**

Branch Prediction Through Loop

```
401029:  mulsd    (%rdx), %xmm0, %xmm0
40102d:  add      $0x8, %rdx
401031:  cmp      %rax, %rdx
401034:  jne      401029
```

i = 98

Assume
array length = **100**

Predict Taken (OK)

```
401029:  mulsd    (%rdx), %xmm0, %xmm0
40102d:  add      $0x8, %rdx
401031:  cmp      %rax, %rdx
401034:  jne      401029
```

i = 99

Predict Taken
(Oops)

```
401029:  mulsd    (%rdx), %xmm0, %xmm0
40102d:  add      $0x8, %rdx
401031:  cmp      %rax, %rdx
401034:  jne      401029
```

i = 100

Read
invalid
location

```
401029:  mulsd    (%rdx), %xmm0, %xmm0
40102d:  add      $0x8, %rdx
401031:  cmp      %rax, %rdx
401034:  jne      401029
```

i = 101



Branch Misprediction Invalidation

```
401029:  mulsd    (%rdx), %xmm0, %xmm0
40102d:  add      $0x8, %rdx
401031:  cmp      %rax, %rdx
401034:  jne      401029      i = 98
```

Assume
array length = **100**

Predict Taken (OK)

```
401029:  mulsd    (%rdx), %xmm0, %xmm0
40102d:  add      $0x8, %rdx
401031:  cmp      %rax, %rdx
401034:  jne      401029      i = 99
```

Predict Taken
(Oops)

```
401029:  mulsd    (%rdx), %xmm0, %xmm0
40102d:  add      $0x8, %rdx
401031:  cmp      %rax, %rdx
401034:  jne      401029      i = 100
```

Invalidate

```
401029:  mulsd    (%rdx), %xmm0, %xmm0
40102d:  add      $0x8, %rdx
401031:  cmp      %rax, %rdx
401034:  jne      401029      i = 101
```

Branch Misprediction Recovery

```
401029:  mulsd  (%rdx), %xmm0, %xmm0
40102d:  add    $0x8, %rdx
401031:  cmp    %rax, %rdx
401034:  jne    401029
401036:  jmp    401040
. . .
401040:  movsd  %xmm0, (%r12)
```

i = 99

Definitely not taken

} Reload
Pipeline

■ Performance Cost

- Multiple clock cycles on modern processor
- Can be a major performance limiter

Branch Prediction Numbers

■ A simple heuristic:

- Backwards branches are often loops, so predict taken
- Forwards branches are often ifs, so predict not taken
- >95% prediction accuracy just with this!

■ Fancier algorithms track behavior of each branch

- Subject of ongoing research
- 2011 record (<https://www.jilp.org/jwac-2/program/JWAC-2-program.htm>): 34.1 mispredictions per 1000 instructions
- Current research focuses on the remaining handful of “impossible to predict” branches (strongly data-dependent, no correlation with history)
 - e.g. https://hps.ece.utexas.edu/pub/PruettPatt_BranchRunahead.pdf

Optimizing for Branch Prediction

■ Reduce # of branches

- Transform loops
- Unroll loops
- Use conditional moves
 - Not always a good idea

■ Make branches predictable

- Sort data
 - <https://stackoverflow.com/questions/11227809>
- Avoid indirect branches
 - function pointers
 - virtual methods

```
.Loop:
    movzbl 0(%rbp,%rbx), %edx
    leal    -65(%rdx), %ecx
    cmpb    $25, %cl
    ja      .Lskip
    addl    $32, %edx
    movb    %dl, 0(%rbp,%rbx)
.Lskip:
    addl    $1, %rbx
    cmpq    %rax, %rbx
    jb      .Loop
```

```
.Loop:
    movzbl 0(%rbp,%rbx), %edx
    movl    %edx, %esi
    leal    -65(%rdx), %ecx
    addl    $32, %edx
    cmpb    $25, %cl
    cmova    %esi, %edx
    movb    %dl, 0(%rbp,%rbx)
    addl    $1, %rbx
    cmpq    %rax, %rbx
    jb      .Loop
```

Memory write
now
unconditional!

Loop Unrolling

- Amortize cost of loop condition by duplicating body
- Creates opportunities for CSE, code motion, scheduling
- Prepares code for vectorization
- Can hurt performance by increasing code size

```
for (size_t i = 0; i < nelts; i++) {  
    A[i] = B[i]*k + C[i];  
}
```

```
for (size_t i = 0; i < nelts - 4; i += 4) {  
    A[i] = B[i]*k + C[i];  
    A[i+1] = B[i+1]*k + C[i+1];  
    A[i+2] = B[i+2]*k + C[i+2];  
    A[i+3] = B[i+3]*k + C[i+3];  
}
```

When would this change be incorrect?

Scheduling

- Rearrange instructions to make it easier for the CPU to keep all functional units busy
- For instance, move all the loads to the top of an unrolled loop
 - Now maybe it's more obvious why we need lots of registers

```

for (size_t i = 0; i < nelts - 4; i += 4) {
    A[i] = B[i]*k + C[i];
    A[i+1] = B[i+1]*k + C[i+1];
    A[i+2] = B[i+2]*k + C[i+2];
    A[i+3] = B[i+3]*k + C[i+3];
}

for (size_t i = 0; i < nelts - 4; i += 4) {
    B0 = B[i]; B1 = B[i+1]; B2 = B[i+2]; B3 = B[i+3];
    C0 = C[i]; C1 = C[i+1]; C2 = C[i+2]; C3 = C[i+3];
    A[i] = B0*k + C0;
    A[i+1] = B1*k + C1;
    A[i+2] = B2*k + C2;
    A[i+3] = B3*k + C3;
}

```

When would *this* change be incorrect?

Vectorization

- **Use special instructions that operate on several array elements at once**
 - Often called “SIMD” for “Single Instruction Multiple Data”
 - Invented in 1966 for ILLIAC IV supercomputer
 - Valuable for audio and video processing; has become ubiquitous

```
for (size_t i = 0; i < nelts - 4; i += 4) {
    B0 = B[i]; B1 = B[i+1]; B2 = B[i+2]; B3 = B[i+3];
    C0 = C[i]; C1 = C[i+1]; C2 = C[i+2]; C3 = C[i+3];
    A[i] = B0*k + C0;
    A[i+1] = B1*k + C1;
    A[i+2] = B2*k + C2;
    A[i+3] = B3*k + C3;
}
```

```
kkkk = _mm_set_ps1(k);
for (size_t i = 0; i < nelts - 4; i += 4) {
    B0123 = _mm_load_ps(&B[i]);
    C0123 = _mm_load_ps(&C[i]);
    A0123 = _mm_fmadd_ps(B0123, kkkk, C0123);
    _mm_store_ps(&A[i], A0123);
}
```

Summary: Getting High Performance

- **Good compiler and flags**
- **Don't do anything sub-optimal**
 - Watch out for hidden algorithmic inefficiencies
 - Write compiler-friendly code
 - Watch out for optimization blockers:
procedure calls & memory references
 - Look carefully at innermost loops (where most work is done)
- **Tune code for machine**
 - Exploit instruction-level parallelism
 - Avoid unpredictable branches
 - Make code cache friendly

Zerowanie górnej połowy rejestrów 64-bitowych

Dlaczego podczas operacji na rejestrach 32-bitowych zerowana jest górna połowa rejestru 64-bitowego?

```
xor %edi, %edi
```

Zerowanie górnej połowy rejestrów 64-bitowych

Dlaczego podczas operacji na rejestrach 32-bitowych zerowana jest górna połowa rejestru 64-bitowego?

```
xor %edi, %edi
```

Czy w poniższym kodzie w liniach 2-3 można zamienić rejestr `%rax` na inny?

```
addq %rax, %rdi  
movq %rbx, %rax  
addq %rax, %rsi  
movq %rcx, %rax
```

Zerowanie górnej połowy rejestrów 64-bitowych

A w poniższym?

```
addl %eax, %edi  
movq %rbx, %rax  
addq %rax, %rsi  
movl %ecx, %eax
```


Zerowanie górnej połowy rejestrów 64-bitowych

A w poniższym?

```
addl %eax, %edi  
movq %rbx, %rax  
addq %rax, %rsi  
movl %ecx, %eax
```

Zerowanie górnej połowy rejestrów 64-bitowych pozwoliło na uniknięcie niepotrzebnych zależności po rozszerzeniu 32-bitowej architektury out-of-order do 64 bitów.